cs page for a number of different jobs. This can help organizations evaluate how they are doing throughout the product development workflow. After implementing these analytics pages, it is difficult to know how useful these are to users and where they use them within their workflows. Are they placed in the most ideal locations for users to efficiently complete their jobs? You will use UX heuristics and our catalog of existing research to determine what these pages do well and what they don't. You'll also be redesigning the pages to make use of Pajamas standards and validating that these new solutions positively impacts the jobs to be done (JTBD) for these pages with real GitLab users. Make sure to work with your team to scope this down so you have enough time to complete this project! For example, focus solely on improving the repository analytics page.
{{% /details %}}

{{% details summary="Do GitLab users understand where to find specific information they need on docs.gitlab.com?" %}}
The GitLab documentation site contains a large and growing amount of content. Findability of specific content has been a problem for users. Recognition and understanding of the different sections of content on the docs site is important for users to be able to find what they·re looking for.

- Do GitLab users understand where to find specific information they need on docs.gitlab.com?
- Do GitLab users understand what types of content might be in each section or "bucket"?
- Do the section names (buckets) meet user expectations?
- If users do not understand the buckets, what are some improvements to increase comprehension?

{{% /details %}}

{{% details summary="Using Continuous Integration (CI) when pushing changes" %}}
CI has become a regular practice for companies around the world since it allows them to develop software easier, faster, and with less risk. Automating builds and tests allows developers to commit smaller changes with higher confidence and get feedback on their code sooner. GitLab's CI features were designed years ago, and given the siloed nature of how GitLab team members work internally, there hasn't been a moment to step back and evaluate the experience of CI and merging code as a whole. In this project, you'll evaluate the current experience, including documentation, and speak with real developers who use GitLab to understand their pains when developing/merging code with CI enabled. You'll use the insights you find to make improvements using the Pajamas design system in Figma and test them with real users.
{{% /details %}}

Project lead

The project lead for the Tufts University Capstone Course project is a rotating volunteer position. Ideally, the same individual should not run the project more than 1-2 years concurrently in order to share the professional development benefits and workload that the role entails.

{{% details summary="Project lead history" %}}

Year	Name
2023	Gina Doyle

2024	Gina Doyle
2025	Chad Lavimoniere
2026	TBD

{{% /details %}}

Responsibilities and timeline

The project lead has various responsibilities before, during, and after the student project.

Before the project starts

In the lead up to the project start, the project lead should:

- Recruit GitLab Team Member volunteers, keeping in mind the ideal volunteer team composition.
- Solicit at least 2 project proposals from GitLab team members for students to vote on.
- Communicate with the Product team DRIs (Product Manager, Product Designer, UX Researcher) that own the feature area for each project proposal.
- Coordinate with the professor, Linda Borghesani (Linda.Borghesani@tufts.edu). She will arrange for the students to choose which proposal.

During the project

At the start of the project, the project lead should:

- Collaborate with Legal to have the students sign an NDA to follow the guidelines of our SAFE framework (:lock: internal only). Once the students select a project in January, it would be best to connect with @ktesh with the project start and end date, a short description of the project topic, and all of the students' emails in order for the NDA to be officially issued.
- Create a designated subgroup and public project in the Tufts University Group to track the Capstone project progress. Encourage students to create GitLab accounts with their school emails and invite them as owners to the project. This can help them get familiar with how GitLab works and the problem they'll be solving. This can also be the place where they access resources or materials they need to complete the project. Check out this project for inspiration.
- Create a Slack channel for daily communication with the students. You can use this Slack Request form to include external users in a Slack channel. You can also use this access request issue as a reference.
- Write up a summary of the selected project proposal and link any resources that would be helpful for the students to look at to gain a better understanding of the concepts they'll be learning. Check out this markdown file for inspiration.
- Schedule a weekly check-in meeting time for volunteers and students
- Create an agenda document for weekly check-ins so that the students know what to expect. Check out this agenda for inspiration.
- During the first meetings with the students, ask them what skills they are hoping to get out of the project. There are opportunities during the weekly check-ins to focus on those skills and teach the students how they are used in real-world scenarios. An example of this is explaining how to use design systems in Figma, including how to get the library in Figma, adding and editing components, and even making contributions to the Figma kit itself. Previous topics that students have asked about are listed in threads on this issue along with supporting

materials.

While the project is in progress, the project lead should:

- Work with the student team to make sure they have access to the necessary resources (for example, Dovetail) and people (for example, experts or participants to interview). Work with your manager to give students the appropriate access to GitLab tools and projects.
- Hold stakeholder review sessions for larger deliverables, such as the discovery research proposal. These meetings should at least include the Product Manager, Product Designer, and UX Researcher of the product area the project focuses on. These can take place instead of a weekly sync or at a separate time depending on the team's preference.
- Use issues in the project to track sponsor-assigned tasks, such as project prep and reviewing student deliverables.
- Record weekly check-ins and add them to the Tufts Capstone Project playlist on Unfiltered. This playlist is private, as some recordings will contain confidential information, such as reviews of confidential research or competitors.
- If the students and professor are interested, organize a final presentation for them in a GitLab-held Zoom meeting. Be sure to:
  - include the Product Department as attendees.
  - Add the event to the GitLab team meetings calendar by following these steps.
  - Post in whats-happening-at-gitlab for extra transparency.

After the project is completed

After the project has been completed, the project lead should:

- Share a write-up of the students' results in the GitLab UX Research project and the gitLab-org/gitlab project. Check out this research summary and feature proposal for inspiration.
- Coordinate with current and past volunteers to establish who the project lead will be for the following year.

Reach out to @gdoyle or @clavimoniere on Slack if you have questions. More information including FAQs can be found here.

SECTION: product/ux/navigation/_index.md

The group::personal productivity team owns the navigation structures of the GitLab product. Please review this information if you plan to propose changes to GitLab navigation.

> Note: a Code Owners approval rule is in place to prevent unapproved changes to the navigation. If you have not followed this process, your Merge Request will be blocked.

What is navigation?

Navigation refers to elements that aid users in moving around GitLab, which includes their organization and wayfinding clues. The navigation experience directly impacts the usability and discoverability of our features. This document describes how we can collectively evolve the navigation while still meeting our goals.

Why do we need to be careful when changing the navigation?

In the past, teams added items to highlight new features. However, this created an overwhelming navigation structure that makes it challenging for users to find what they need. Our quarterly GitLab SUS survey continues to highlight these recurring themes:

The navigation is complex or confusing
The navigation is difficult to learn
The navigation is not intuitive

What kind of navigation changes require approval?

While GitLab's UX allows users to navigate across the product in different ways, the focus of the approval process is on the left-side navigation. This area serves both as navigation and as a feature discovery point for users. As the navigation evolves, it's crucial that we maintain a balance between a focus on core workflows and feature visibility.

To help maintain this balance, we ask for everyone to use this process when proposing changes to the left-side navigation:

First, second, and third-level navigation additions
Renaming a navigation item
Removing a navigation item
Changing the sort order of navigation items
Changing navigation functionality or features
Launching an Experiment or Beta feature
Changing the viewership of a navigation item (e.g. moving from disabled by default to enabled by default)

When to change the navigation

We only make new additions to the GitLab navigation structure through a deliberate process that is intended to optimize user workflows. This video summarizes the main factors that are important to consider as we iterate on our navigation. In the past, teams added items to highlight new features. However, it becomes impossible to accommodate every new feature, as this creates an overwhelming navigation structure that makes it too difficult for users to find what they need to complete their tasks.

Therefore, we do not add new items to:

Improve discoverability of new features. Instead, look for other opportunities to highlight the functionality throughout the product.
Optimize for the potential future. We should be forward thinking without over optimizing. As features are developed and added, we can look into what changes may need to occur to support a growing feature.

How do I evaluate navigation changes?

There are two main questions we need to answer for navigation changes:

1. Does this navigation proposal facilitate one of our primary JTBDs? Which job?
1. How does this change improve the workflow for users attempting to complete that job?

There are many different types of Problem Validation research that could be done to learn about areas of opportunity for our navigation. The specific type of research performed should be based on the research questions and goals of the study. The following research methods and frameworks are some examples of ways research can be used to justify a new addition to the existing GitLab navigation:

1. Contextual inquiry: This method involves observing and interviewing users as they perform tasks related to their role to understand the "why" and the "how" behind them.
    Example: Users may demonstrate how they navigate through multiple menus and pages in GitLab to find their dashboards. This insight would reveal the need for a single page in GitLab to find all dashboards.
1. Jobs to be Done (JTBD): This framework is a type of foundational research meant to learn the job(s) that customers want to accomplish within their roles. Jobs allow us to define the circumstances, goals, and outcomes of customers' work.
    Example: By interviewing users, we learn that have a main job of tracking metrics to show the health of their application, so they can identify when the application is failing. This insight would suggest a gap we could fill in GitLab by adding a page for displaying application metrics.
1. Diary study: This method is used to gain feedback from users over a period of time, so we can uncover changes that occur over days, weeks, or months.
    Example: When gathing feedback about how users engage with a major change to GitLab's navigation bar, we observe that GitLab admins continuously struggle from one month to the next with locating the admin area to adjust settings. This insight would suggest we need to surface the admin area in the user interface since it is not easy to find over time.

Questions to ask to learn whether a change to the navigation is needed:

1. What are the main tasks assocaiated with your current role?
1. Why are those tasks important? What would happen if you could not complete those tasks?
1. Show me how you perform a specific task using your existing tools.
1. What, if anything, would you improve about the process of completing that task?

After there is insight into a problem with the navigation, the Product team DRI should work with Product Design and UX Research to evaluate the ideal solution through a subsequent solution validation study.

How to propose a navigation change

> If your primary goal is to improve discoverability of your feature, please start by looking for other opportunities to highlight the functionality throughout the product.

1. Before opening an issue, review the elements and patterns for navigation in Pajamas. It is worth checking the direction page to see how your proposal aligns or conflicts with upcoming changes.
1. Review the list of navigation changes and what they are to make sure your change qualifies.
1. The Product Manager for Personal Productivity is the DRI for navigation changes. Reach out to them to determine whether your proposal needs full validation or limited validation.

1. You can initiate the review for this process by using the Navigation Proposal issue template.
1. Designers on Personal Productivity will assist the DRI by reviewing the proposal and providing input. The typical turnaround time from the Personal Productivity team will be 1 milestone. After providing feedback, it is the responsibility of the DRI to move the proposal forward and seek additional feedback as needed.
1. When you have approval and are ready to start implementation, then follow the GitLab Docs on adding items to the navigation.

Full validation path

This path is suitable for navigation changes that affect a majority of GitLab users or that introduce new design patterns. Some examples of changes that may need full validation are:

Launching a Beta or Generally available feature
Changes to navigation structure or functionality
Removing an existing navigation item
Renaming an existing navigation item

The Personal Productivity PM is the DRI for determining if your proposal should follow the full validation path. On this path, we require the following steps be completed and documented as part of the navigation proposal issue.

Step	Requirement
Business case	All proposals must include a business case that identifies the underlying problem and goals of changing the navigation.
Problem validation	Problem validation research is required to discover and verify the areas of opportunity for our navigation.
UX review	A UX review has been conducted with the design DRIs for the related stage group.
Counterpart support	The proposal is supported by product, design, and research counterparts for the related stage group.
Solution validation	Solution validation research is conducted to evaluate a navigation change against other potential solutions.

Limited validation path

This path is suitable for navigation changes that affect a minority of users or that follow pre-existing design patterns. Some examples of changes that may need limited validation are:

Experimental features available behind a feature flag
New 3rd party integrations that follow the pattern of existing integrations
Changes that bring consistency where there is already inconsistency

The Personal Productivity PM is the DRI for determining if your proposal can follow the limited validation path. On this path, we require the following steps be completed and documented as part of the navigation proposal issue.

| Step | Requirement |

| --- | --- |
Business case	All proposals must include a business case that identifies the underlying problem and goals of changing the navigation.
Problem validation	User research is not required to identify the area of opportunity for our navigation.
UX review	A UX review has been conducted with the design DRIs for the related stage group.
Counterpart support	The proposal is supported by product and design counterparts for the related stage group.
Solution validation	Proposals need to describe or visualize alternative solution(s) that surface changes at other points of the user journey. Then critically assess how well the proposed and alternative solutions address the problems and goals outlined in the business case.

Reconciliation process

The navigation proposal process attempts to balance a focus on core workflows and feature visibility, which means sometimes proposal authors may disagree with the Personal Productivity team decision. If you feel like we've struck the wrong balance, let's follow the manager mention thread process. Add a comment to the proposal and:

1. Summarize why the proposal was rejected and your perspective on why this decision is incorrect
1. Mention your manager and the Personal Productivity PM manager
1. Request their input on the decision

SECTION: product/ux/navigation/inventory.md

<!--more-->

How to read this document

This document contains an exhaustive inventory of the GitLab product navigation, dynamically generated from the gitlab-org/gitlab repo. Each major product context is presented in its own section. Note that some menu items only appear in certain environments. Those entries are tagged with the following labels:

{{< label name="ff" >}}	Behind a feature flag
{{< label name="sm" >}}	Only on self-managed installations
{{< label name="dotcom" >}}	Only on SaaS instances

Product contexts

{{% product/navigation-inventory %}}

Missing a nav item?

Open an issue in gitlab-org/gitlab describing the context, group, and name of the missing item. Include instructions on how to enable that item in the nav (e.g. set up X integration or enable Y setting). Add the {{< label name="group::personal productivity" color="A8D695" light="true" >}} label and we'll pick it up in our next issue triage cycle.

Updating this page

1. Generate the latest navigation information from the gitlab-org/gitlab repo. See Development Rake tasks for instructions.
1. Copy the generated navigation.yml into handbook/data/navigation.yml
1. (Optional) Rebuild the handbook locally to verify the output of this page
1. Open an MR with the updated navigation.yml content. Assign it to @jtuckergl.

SECTION: product/ux/pajamas-design-system/_index.md

What is the Pajamas Design System?

Located at design.gitlab.com, the goal of Pajamas
is to be the single source of truth for the robust library of UI components that
we use to build the GitLab product, including usage and implementation guidelines.

As a fully implemented design system, Pajamas includes:

- UX/Design Philosophy: GitLab has strong foundational principles by which
we work, and Pajamas reflects those values—helping to tell the story of
how we build products and translating "our way of working" to the outside world.
- Contribution Guidelines: We clearly outline how anyone can contribute to
GitLab through issue creation/discussions, component documentation, designs,
and/or code.
- Component Documentation: Every component includes guidelines for Interaction
(how it works), Visual (how it looks), and Usage (how you should use it).
- Page Layouts: We clearly document page layout options and our grid implementation.
- Content Guidelines: We provide voice and tone guidance for our UI that
aligns with GitLab's brand standards.
- Accessibility Guidelines: As a company, GitLab strongly believes that
everyone should be able to contribute. That means we must always consider how to
design and code in an adaptive way to support our users, regardless of their abilities.
- Developer Resources: Our components include live code snippets and developer
documentation for our Vue.js based front end.
- Designer Resources: We provide tools to make designing for GitLab easier—for
example, our Sketch pattern library.

Why is Pajamas important?

Pajamas enables anyone to contribute towards GitLab. It allows Product, Engineering,
UX Design, and Contributors to work together more seamlessly and improve our product faster.

It's efficient.

- Product Designers can spend more time solving problems and less time designing (and redesigning) UI components. Components can be reused, making design efforts scalable and ensuring our UI stays DRY.
- Engineers can reference design documentation that enables them to easily eliminate inconsistencies between design and code without assistance from a Product Designer.
- Engineers can find coding and development guidelines which will enable them to write better code and conform to set practices more efficiently.
- Product Managers can quickly propose solutions that follow documented usability guidelines.

It creates a cohesive product.

- GitLab Teams can make fast, continuous feature additions within their iterative groups, while still maintaining consistency with other product areas.
- GitLab Users can get up to speed more quickly on new features, because they don't need to spend time learning how to interact with inconsistent UI patterns.
- Product Designers can feel more confident that their designs are visually appealing, consistent, and on brand.
- It enables Marketing and UX to create consistent experiences across GitLab through a shared visual language.

It helps GitLab communicate.

- We can more quickly envision and align on the details of how a proposed solution might be implemented.
- It proves to our customers and Contributors that we have a deep commitment to improving the experience of our UI.
- It improves alignment between departments through shared principles and personas.

Who can contribute?

Everyone can contribute! Pajamas is a living system, meaning it continually evolves to support new UI components and also deprecate outdated components. For this evolution to be scalable, we encourage everyone to contribute ideas, designs, code, and bug reports.

External contributions can be particularly valuable, because GitLab Contributors have experiences and insights we may not have considered.

We have specific rules around design review.

When do we add a new component to Pajamas?

Not every component used in the GitLab product must be codified as part of the design system, because sometimes we create components that are relevant for only a specific use case in a distinct product area.

To learn more about when to add a new component to Pajamas, read our component lifecycle documentation.

What is the component development lifecycle?

The goal of this process is to make it easy to: submit new designs (including documentation), propose changes to existing designs, and translate component designs into built components.

To learn more about the stages of the component lifecycle, read our component lifecycle documentation.

Beautifying the GitLab UI

A design system is only valuable if it's used consistently throughout the product it supports. Because Pajamas is a new design system, we have to catch up the existing product to using "single source of truth" Pajamas components.

To ramp up implementation, we commit to ensuring that:

- Building New Components: Starting in the 11.11 milestone, each stage group commits to completing one Vue.js component per release within gitlab-ui, including correct functionality and styling. The Product Designers in each stage group will collaborate with the PM and Frontend Engineering Manager to determine which component and how to prioritize.

How do we measure success?

There are two avenues of success in relation to a design system: adoption and goal achievement. Providing metrics for both adoption and goal achievement provides the organization with a tangible way to measure the overall success of a design system over time.

Adoption

The value of a design system is only fully realized when the organization ships product that uses its parts. A commitment to adopt is essential to ensuring that the design system achieves the goals highlighted in this document.

To measure adoption, we:

- Track the status of individual components.
- TBD

Goal Achievement

As design system adoption increases, we are able to measure the success of our goals, which can be evaluated regularly at major adoption milestones (0%, 25%, 50%, 75%, 100%).

- The UX team sends surveys to Product Designers and Front-End Engineers to gather data related to the time they spend building unique components.
- We pull reports to determine the number of UI polish and

Deferred UX
issues to track trends over time. A reduction in overall issues affirms consistency throughout the product.

SECTION: product/ux/pajamas-design-system/design-review.md

Overview

The Pajamas Design System is composed of various projects, where design reviews are mandatory:

1. design.gitlab.com
1. gitlab-svgs
1. gitlab-ui

This page describes the roles of reviewer and maintainer in approving and merging your (or a community member's) merge request (MR). It also describes the process for becoming a maintainer.

Reviewer

All GitLab Product Designers can (and are encouraged to) perform design and code reviews on MRs that impact Pajamas. This includes contributions from GitLab Team Members and the wider GitLab community. If you want to review MRs, you can wait until someone assigns you one, but you are also more than welcome to browse the list of open MRs and leave any feedback or questions you may have.

To perform a review, you should familiarize yourself with and follow:

- Our general Code Review guidelines.
- Our general MR review guidelines for Product Designers.
- The contribution guidelines for the Pajamas projects.

Note that while all designers can review all MRs, the ability to accept MRs is restricted to maintainers. You can find all design reviewers and maintainers on the list of GitLab Engineering Projects.

Maintainer

Maintainers are GitLab designers who:

- Are experts at design and code review, including reviewing commit messages.
- Know the GitLab product, design guidelines, and code base very well.
- Are empowered to accept MRs in one or several of the Pajamas projects.

Every project has at least one maintainer, but most have multiple, and some projects (like gitlab-ui and design.gitlab.com) have separate maintainers for design and frontend. As with reviewers, design maintainers can be found on the list of GitLab Engineering Projects.

Read more about what makes great maintainers in the Engineering Review Workflow.

Maintainer types

Design maintainers are divided into types of maintainership. This helps maintainers specialize, but more importantly, it helps speed up the process of becoming a maintainer, and with knowing who to ask for final acceptance of MRs. New types can be created or existing ones refactored to accommodate the natural evolution of projects.

Project	Maintainer type
design.gitlab.com	Figma (Pajamas UI Kit): reviews file organization, object properties, interaction design, accessibility, visual design, and technical feasibility. UX (Pajamas website): reviews content meaning, terminology, and structure across all sections of the website.
gitlab-svgs	Figma (Pajamas UI Kit): reviews icon and illustration file organization, object properties, and visual design.

If you are interested in becoming a Maintainer of UI (.scss) for the gitlab or gitlab-ui projects, please follow the Engineering Review Workflow.

How to become a maintainer

We follow the same maintainer guidelines as our Engineering counterparts. Get familiar with those guidelines and how to become a maintainer in the Engineering Review Workflow.

Three key aspects of that process:

1. "Maintainer-level" MRs: Candidates should have specific examples of recent "maintainer-level" MRs. They can work on any kind of MR, but "maintainer-level" ones are the focus of the maintainership. "Maintainer-level" MRs are described in the Engineering Review Workflow.
1. Reviews or contributions: Contributing MRs also counts. Whether MRs are reviewed or contributed by a candidate, they should consistently make it through reviewer and/or maintainer review without significant required changes.
1. Traineeship optional: The trainee maintainer program (traineeship) supports reviewers in becoming maintainers, but the program is not a requirement. Designers that have been recenly involved in a fair amount of "maintainer-level" MRs can become maintainers without the traineeship. Anyone can nominate oneself (or someone else) for maintainership, following the process described in the Engineering Review Workflow.

Trainee maintainer

We're not able to support more trainees at the moment. We can only accommodate trainee's once we have a Support Maintainer available and there is a need for additional maintainers. If you have an interest in becoming a maintainer, we encourage you to talk with your manager!

Note: While maintainers are responsible for certain projects, becoming one is not required for career progression and this should not be the primary reason for becoming a trainee.

- We follow the same trainee maintainer program (traineeship) as our Engineering counterparts.

Anyone may nominate themselves as a trainee by opening a tracking issue using the Trainee design maintainer template.

- For the traineeship, a design maintainer is assigned as Support Maintainer to each trainee. The Support Maintainer will direct MRs for review, give feedback on proposals, and discuss process or progress during 1:1 sessions. Trainees can always reach out to their managers or other maintainers for feedback and guidance separate from the dedicated Support Maintainer. We've also created a Pajamas UX maintainer review checklist to help guide trainee maintainers as they learn to review Pajamas MRs.

- We try to keep a maximum of one trainee per maintainer, so that maintainers are not overloaded with their additional role as Support Maintainers and can provide adequate guidance. See our current trainee maintainers.

Duration

The traineeship is a long commitment, usually several months, and takes away time from other responsibilities. If you're interested in enrolling in this program, please talk with your manager and team before nominating yourself, as the traineeship is likely to impact your capacity.

There are two aspects that play a big part in the duration of the traineeship: the number of hours that are dedicated to it and the number of available MRs for the trainee. When these two aspects oppose each other, the traineeship can take longer than expected:

1. Many hours, few MRs: To increase the number of MRs, the trainee can always make their own contributions. Reviewing MRs from others is not the only way to become a maintainer. The trainee must be creative and try to work on "maintainer-level" MRs as much as possible.
1. Few hours, many MRs: Trying to review or contribute many MRs in few hours can have a negative effect on quality. The trainee should focus on quality because that is what is evaluated. They should also follow our review-response Service-level Objective (SLO). If the trainee wants to speed up the traineeship, they should talk with their manager to find ways to balance their workload and free up more time for this program.

To help track progress, we encourage trainees to make the traineeship one of their personal OKRs.

Maintainer ratios

See the Pajamas maintainer ratio dashboard (internal).

Current trainee maintainers

Project	Trainee	Support Maintainer
Pajamas Design System (Figma) - PAUSED	Michael Le	Jeremy Elder

---
SECTION: product/ux/performance-indicators/_index.md

To view the charts, you must sign in to Tableau. GitLab team members only.

{{% performance-indicators "uxdepartment" %}}

// want to edit? Go here --> https://gitlab.com/-/ide/project/gitlab-com/www-gitlab-com/edit/master/-/data/performanceindicators/uxdepartment.yml

---
SECTION: product/ux/performance-indicators/system-usability-scale/_index.md

The System Usability Scale (SUS) is a standardized metric used to measure usability perception of computer interfaces. Our current and past SUS scores can be found in the UX Department Performance Indicators.

Collectively, SUS consists of ten questions, which are positively and negatively oriented:

1. I think that I would like to use GitLab frequently.
1. I found GitLab unnecessarily complex.
1. I thought GitLab was easy to use.
1. I think that I would need the support of a technical person to be able to use GitLab.
1. I found the various functions in GitLab were well integrated.
1. I thought there was too much inconsistency in GitLab.
1. I would imagine that most people would learn to use GitLab very quickly.
1. I found GitLab very cumbersome to use.
1. I felt very confident using GitLab.
1. I needed to learn a lot of things before I could get going with GitLab.

The response scale for each question is a 5-point Likert agreement scale:

Strongly disagree	Disagree	Neither agree nor disagree	Agree	Strongly agree
1	2	3	4	5

We follow these 10 questions with a single open-ended question that asks, "What problems or frustrations have you experienced while using GitLab?"

These questions are delivered in survey format to users of the product.

SUS and GitLab

We adopted the System Usability Scale at GitLab in FY20-Q1. We deploy the survey bi-quarterly to both our SaaS and Self-Managed users. This has allowed us to understand the overall usability of our product and track changes over time. We've begun relying on SUS as a KPI for the UX Department and we have multiple OKR-related efforts underway to try and improve our score.

With this emphasis, it's important that SUS is deployed in a rigorous and sustainable manner.

Executing SUS

We track the SUS Survey results in GitLab where there is an epic of the current results of the System Usability Scale (SUS) Survey.

To ensure that our SUS metric can be reliably and sustainably collected, and that it accurately represents our user population, in FY21-Q4 we developed a new method for collecting SUS with the following requirements:

- Sustainable: We can sample a sufficient number of users without exhausting our userbase. A user should not receive a SUS survey invite more than once per year.
- Segmentable: We pre-define the segments of users we care about, and we collect a sufficient number of responses for any segment for which we want to derive a score (but only those predefined segments). If we want to alter the segments we're targeting, we can modify those in future quarters.
- Verified criteria: We'll pull that latest data from the data warehouse or Marketo to identify appropriate users, including things like a user's plan and account age.
- Active users: We can target users who we know have recent experience with the product, including minimum usage within a certain time period or usage of multiple stages.
- Randomized: Other than the criteria we define, we should feel confident the users we invite to respond do not skew in any particular direction, such as geographic location or organization size.

Regular participant criteria

- Recently active: We use a minimum threshold of 10 product events across at least 2 stages in the previous 30 days. An 'event' is an indicator that users are doing something in a certain area of GitLab. This approach has two goals: we're targeting people who have used multiple stages, and eliminating people with limited exposure to our features and the usability of our experience. It also ensures respondents have recently used GitLab and have a higher likelihood of experiencing recent improvements.
- Sample size: For SaaS users we target an n of 200 for each cohort with a total n of 800. This allows us to calculate a score with a high degree of confidence.

Regular cohorts

We have defined the following cohorts that we will track over time:

- Paid users: Users that are associated with a paid subscription (whether that subscription was purchased or gifted by GitLab)
- Free users: Users that are not currently associated with any paid subscriptions and are using the Free plan.
- Experienced users: Users that have a tenure of 180 days or more.
- New users: Users with a tenure of less than 180 days.

Note that these cohorts will overlap, so we won't necesssarily be gathering 200 responses for each one. For example, an experienced free user would be considered part of both the Free user cohort and the Experienced user cohort, and would be counted for both of those quotas.

Self-Managed cohort

In order to understand how the experiences of our Self-Managed users compare to those of our SaaS users, we conduct a limited SUS measurement of Self-Managed users every other quarter. The majority of these users are recruited via Marketo.

The Self-Managed cohort has the following criteria:

- Self-Managed user: Users self-report that they are a user of a Self-Managed instance of GitLab.
- Recently active: Users self-report that they have been active on a Self-Managed instance in the last 30 days.
- n = 100: Given that Self-Managed users are harder to recruit for, we want to lower our sample size as to not exhaust Self-Managed users in the Marketo panel and to acquire data in a timely fashion. This cohort will have a higher margin of error compared to our regular cohorts.

Distribution

We distribute survey invites via email using Qualtrics, our survey tool. Responses are incentivized using a sweepstakes (Promotional Games workflow). Those that complete the survey are entered into a quarterly sweepstakes to win one of three $200 gift cards. The Self-Managed cohort is incentivized at the same rate of three $200 gift card winners in order to maximize the response rate of the limited population.

We aim to start sending email distributions every couple of days in the last month of the quarter until we achieve our desired sample size.

To ensure respondents meet our activity criteria, we pull a new user list for every email distribution. Anyone who has been sent a SUS survey in the previous 12 months is scrubbed from the list.

Analysis & Reporting

We summarize results and calculate scores for SUS on a fiscal year quarterly basis. For each quarter, we report an overall SUS score and SUS scores for any predefined segments. We also report average scores for each individual question for those same segments. All reports and results are stored internally in the System Usability Scale folder (internal only).

Calculating SUS scores

SUS is scored on a 0-100 scale. We normalize the scores for each question. For positive-oriented questions, we subtract one from the original score. For negative-oriented questions, we subtract the original score from five. This gives a consistent scale for all responses of 0-4. We then add up the scores for all questions and multiply that sum by 2.5 to get our final score. For example, if the sum of a user's responses to the ten questions is 30, we'd multiply that sum by 2.5 to get a SUS score of 75.

Calculating scores for the individual questions that make up the SUS is not part of the standard process. We do it to gain additional directional insight into how we're doing with specific aspects of the experience. For example, if the average response for the question about

the system being inconsistent is lower than the average, this tells us that inconsistency is a problem we should investigate further.

Our individual question scores mirror the scale used for the overall SUS score. This allows us to understand how individual questions are performing relative to the overall score. To get this score, we take the normalized single question score and multiply it by 25. For example, if the average response to a single question is 2.5, we then multiply that average by 25 to get a SUS-equivalent score of 62.5.

Interpreting SUS scores

Even though SUS scores are on a 100-point scale, they are not percentages and should not be treated as such. Jeff Sauro recommends communicating SUS scores as percentiles. We include a percentile and letter grade using the proprietary scale Sauro developed whenever we report a SUS score.

Company targets for our SUS score are: 73 by Q4-FY24, 77 by Q4-FY25, and 82 by Q4-FY26.

Grade	SUS	Percentile	Adjective
A+	84.1-100	96-100	Best Imaginable
A	80.8-84.0	90-95	Excellent
A-	78.9-80.7	85-89	
B+	77.2-78.8	80-84	
B	74.1 · 77.1	70 · 79	
B-	72.6 · 74.0	65 · 69	
C+	71.1 · 72.5	60 · 64	Good
C	65.0 · 71.0	41 · 59	
C-	62.7 · 64.9	35 · 40	
D	51.7 · 62.6	15 · 34	OK
F	25.1 · 51.6	2· 14	Poor
F	0-25	0-1.9	Worst Imaginable

Communicating out the themes

Every quarter, we review feedback from survey respondents and code the responses into high-level themes. We use these themes to highlight trends over time and gain a deeper understanding of the areas that are impacting the usability of GitLab. The survey verbatims and corresponding list of themes are first shared in a Google spreadsheet so that all team members can access the data. We encourage product managers and product designers to review the feedback and search for existing or related issues in the GitLab issue tracker.

SUS verbatims share out by stage

Every quarter, an issue will be created (see issue template) and assigned to the PDMs to categorize the verbatims that fall into their stage(s). Once the verbatim is categorized by stage, it will automatically get populated in the stage-specific tab within the SUS document. The stage-specific verbatims then will be distributed via designated Slack channels.

| Stage | Slack channel(s) to communicate findings UXR |

|---|---|
Manage	smanage
Plan	splan
Create	screate
Ecosystem, Foundations, Integrations	gmanageintegrations, gmanagefoundations
Verify	sverify, opssection
Package	spackage, opssection
Release	genvironments, opssection
Configure	genvironments, opssection
Monitor	smonitor, opssection
Secure	ssecure
Software Supply Chain Security	ssoftware-supply-chain-security
Growth	sgrowth
Fulfillment	sfulfillment
Enablement	senablement
ModelOps	smodelops

Use the following sample messaging text when sharing out the stage-specific insights:

markdown
Hello :wave: - We just completed analyzing the Q FY System Usability Scale (SUS) data! I wanted to share the verbatim that's relevant to us in the fill in stage name here stage. Here's a sampling:

Stage UXR to paste example in italics
Stage UXR to paste example in italics

You can view the remaining quotes here --> link to the Sheet, on the specific tab

Let me know if you have any questions or if you're interested in pursuing some of these!

SUS Responder Outreach

We include a question at the end of the SUS survey that asks whether respondents would be interested in discussing their responses with a GitLab team member. We have created a process for Product Managers (PM) and Product Designers (PDs) to conduct follow-up interviews so that they can get a better understanding of the usability issues respondents are experiencing. This outreach is optional for team members but highly encouraged. Learn more about the process on the SUS Responder Outreach page.

Limitations

We can only cut by the segments we predefine

It's natural to try and slice survey response data by every facet imaginable to try and find unexpected insights. However, if we don't have a large enough sample size for that particular slice, we might calculate a score in which we don't have high confidence, and that could even be misleading. This is why we define the segments we want to understand before we distribute our survey, so we can ensure we hit our quotas and collect a sufficiently large sample for each

of those segments. This allows us to have high confidence that a score accurately represents a given segment.

SUS is a lagging, incomplete indicator

We deploy SUS on a regular basis, but that doesn't mean we should expect to see improvements reflected in the score immediately. Once we ship a product change, people first have to experience it in sufficient numbers such that our survey reaches enough of them. This can take different amounts of time depending on the usage of a given feature and is effectively impossible to estimate. We also have no way of knowing what the effect of single product change will be on a user. Something that is a major pain for a large number of users may be a minor annoyance for the person we survey. We shouldn't expect single enhancements to drive increases in the SUS, but rather, that sustained enhancements over time will lead to improvements to our overall usability, which in turn should increase our SUS score over the long term.

Frequently Asked Questions

Q: Can we calculate a SUS score for a particular stage or feature?

A: SUS is meant to be a measure of the usability of GitLab in an overall sense. Our users do not necessarily conceptualize the product as a collection of stages. The questions we ask are also framed at an overall level, and we do not ask about their usage or feelings about any particular features. Thus, calculating a score for a particular stage assumes that the responses people give reflect their usage of a certain stage when we have no way of knowing that for certain.

Q: Can we calculate a SUS score for a certain segment of users?

A: To calculate a score for a particular segment of users (such as a certain plan type), we must have a sufficient sample size to ensure we can be confident our score is valid and not the result of random chance. When we want to calculate a score for a subset of users, we predefine it before starting collection. This allows us to modify our recruiting criteria to ensure we collect enough responses.

Q: What is a sufficient sample size to be able to calculate a score for a subset of users?

A: There isn't a single answer to this question, as sample size depends on how many people in the overall population fit your criteria. The larger the overall population, the smaller the sample size needs to be as a proportion of that population. You can use a sample size calculator to estimate your size requirements. Sometimes we can satisfy sample requirements using people we've already surveyed if they meet the additional criteria. Other times, we will need to specifically identify and target users to meet our sample requirements.

Q: How can I find open issues that relate to SUS?

A: Issues that relate to SUS will have one of the labels indicated in the How we use labels section of the UX Department Handbook. If you see an issue that relates to usability problems that fall in line with recent SUS findings, feel free to add one of the labels to it. When in doubt, reach out to your manager or ask in the ux Slack channel.

SECTION: product/ux/performance-indicators/system-usability-scale/sus-outreach.md

Every quarter, we include a question in the SUS survey that asks whether respondents would be interested in discussing their responses with a GitLab team member. This is a great opportunity for Product Designers and Product Managers to learn more about the usability challenges for their specific product area.

Overall process

1. The UX Researcher running the SUS survey shares a Google sheet that contains scores and verbatims from respondents who have agreed to a follow-up conversation.
1. The UX Researcher creates an outreach issue to manage the responder outreach.
1. First, Product Leaders go through the Google sheet and sign up for outreach and/or tag in their Product Managers.
1. Product Managers review the sheet and confirm who they want to speak to. If a Design counterpart exists, tag them Product Design counterparts to decide who will be leading the interview. Otherwise the PM will lead it. The interview leader will be responsible for reaching out to the user and scheduling the interview. Once contacted, they mark the respondent as 'Contacted' in the sheet.
1. If multiple GitLab team members are interested in speaking to the same respondent, coordinate with the team member that contacted the respondent to align on questions to be asked. We do not want the same respondent be contacted multiple times and by multiple GitLab team members.
1. After the interview, mark the respondent as 'Complete' in the sheet.
1. Add any notes and links to video recordings to the UX Research shared Google Drive.

- Current SUS UX Researcher: @nickhertz

Process for reaching out to respondents

1. Calendly is the best method for scheduling calls with respondents. Set up your free Calendly account if you haven't done so. Add details to the invite description describing yourself and the conversation purpose. Also add your personal Zoom link, either via connecting your Zoom account or pasting in your personal Zoom URL.
1. You'll need to add an extra question to the invite form in order to ask for permission to record (example below).
1. Draft an email that you'll send to respondents. Example copy is below.
1. BE ON TIME TO YOUR CALL. Make sure everyone on the call introduces themselves.
1. If respondents have agreed to recording, still ask them once again if it's OK if you record before turning it on.
1. See our training materials on facilitating user interviews

Example email copy:

> Hello,
> My name is X and I'm the Product Designer for X at GitLab. Thank you for giving us the opportunity to follow up on your response to our recent survey.
>

> I would be very interested in speaking further about some of the points you raised in your survey response. Would you be willing to do a 30-minute video conference call to give us some more detailed feedback on your experience using GitLab? You'd be able to schedule the call at a time convenient to you.
>
> Schedule a time for the call using this link: Insert Calendly link here
>
> Thank you for your feedback and let me know if you have any questions.
>
> Best,
> Your name

Copy for the permission to record question in Calendly invite:

> We would like to record (1) the conversation you have with our team member and (2) anything you choose to share on your screen with us during the call. We would like to share the recording of the call with other GitLab team members. The recording is stored securely and would never be shared with anyone outside of the GitLab team. Please indicate below whether you give your permission for us to record the conversation and share with our team.
>
> I agree, I give my permission for my voice and screen to be recorded.
>
> I disagree, I do not want my voice or screen to be recorded.

Before the call

1. Locate and review the SUS Interview Template in Dovetail. Make sure to copy this template into a new note file for your session.
1. Update the note to include information about your session. Specifically, copy over the participants' SUS score and verbatim from the survey, so you can learn more about the user's feedback.

After the call

1. If multiple GitLab team members are on the call, it can be beneficial to debrief immediately afterwards.
1. Collect all notes that were taken and the upload the Zoom recording (if applicable) to our SUS Follow-up Project in Dovetail.
1. If you told the user you'd follow up on anything or promised to send them further information, make sure you do so, ideally within two business days.
1. Add the label 'SUS::Responder Outreach' to any epics/issues that result from or align with feedback from the SUS outreach calls.

SECTION: product/ux/performance-indicators/usat/_index.md

Within the Product division, we have adopted Product Customer Satisfaction Score (PCSAT) and are conducting this survey on a quarterly basis.
We previously surveyed users with the Net Promoter Score (NPS) metric, which we moved away

from starting in FY25 Q1 (see internal only proposal deck).

We are using USAT because it allows us to:

- Measure satisfaction directly vs. indirectly.
- Connect satisfaction ratings and open ended responses back to changes in our product.
- Compare and contrast USAT survey data against our other company wide metrics (System Usability Scale (SUS)).

Satisfaction surveys across GitLab

There are two teams across GitLab who run separate, but related satisfaction surveys. UX Research conducts the User Satisfaction (USAT) survey and Customer Success conducts the All-Customer Satisfaction survey. This handbook section has more details on how these two satisfaction surveys are different.

How the USAT survey is run

UX Research determines our USAT score for paid users of GitLab.com and self-managed GitLab on a quarterly basis through a survey launched through Qualtrics. A USAT template issue.md?reftype=heads) is created by the UX Research DRI at the beginning of each quarter. The issue template contains background information, research goals, and processes on conducting the survey from start to finish. Data is collected over a period of four weeks starting in the beginning of each quarter and the survey stays open until we have met our data collection goals. The next two weeks after data collection are used to clean, analyze, and report on the survey responses. All documents created are stored in an internal only Google Drive within UX Research.

USAT questions

1. How satisfied are you with GitLab (the product)?
   - Very dissatisfied
   - Dissatisfied
   - Neutral
   - Satisfied
   - Very satisfied
2. Why are you satisfied or dissatisfied with GitLab (the product)? (optional)
   - Note: Open text field
3. How could we improve your satisfaction with GitLab (the product)? (optional)
   - Note: Open text field
4. Would you be willing to talk with someone at GitLab about your feedback? This would be for research purposes, not for sales or marketing.
   - Yes, I would be willing to discuss my feedback (please enter the email address where you would like to be contacted)
   - No, I would not like to be contacted

Current workflow

Sampling goals

UX Research aims for a sample size of 384 total responses with plan type proportions for our sample to be +/- 3% compared to the population proportion. Self-managed users are excluded from

this proportion because we've traditionally had difficulty recruiting them for research. When we do capture responses from self-managed users (roughly once a year), we aim for 50-100 responses, which is based on the number of self-managed user responses to past System Usability Scale surveys.

Data analysis

Data analysis consists of two components: quantitative analysis and open response coding.

1. Quantitative analysis: The majority of the quantitative analysis is accomplished by using the analysis template. The final cleaned data inserted in the completed tab automatically updates information in the summary tab, which shows the overall results. From there, the UX Research DRI can transpose the results from the summary tab and insert them into the USAT Google Slide deck template.
2. Open response coding: The UX Research DRI will review the open response feedback left by USAT respondents in the open ended feedback tab of the analysis template and categorize the data into a list of pre-defined themes (see theme glossary tab for descriptions and examples for each theme).

Deployment

Generating a list of eligible users

At the beginning of each quarter, the UX Research DRI will generate a list of eligible users to distribute the USAT survey through the following steps:

1. Generate a new list of paid GitLab.com users to contact by using the following query in Snowflake. Query for approximately 40,000 user IDs and email addresses. Note: You will need access to Snowflake and SAFE data to query the data tables in this step. If you do not have access to one or both, please create an Access Request issue.
2. Contact the Research Operations Coordinator through a recruiting request issue template to generate a list of paid self-managed GitLab users.
3. Create a list of users you've compiled in steps 1-2 within the USAT user Google Sheet template to track contacts for the quarter.
4. Remove users that were contacted more than 12 months ago from the list of previously contacted users.
5. Create a copy of the sheet containing the users you will contact this quarter and insert it in the CSM / UX Survey Participants Google Drive Folder. Contact the Customer Success team, so can avoid contacting the same users in their Customer Satisfaction (CSAT) survey.
6. Calculate the proportions by plan type for the current quarter. Add these percentages to the user list to help calculate how many users for each plan type you need to invite. The end goal is to achieve a sample breakdown that roughly matches the population's breakdown (+/- 3%). Note: You will need access to Tableau and SAFE data to view the link in this step. If you do not have access to one or both, please create an Access Request issue.
7. Create a new project in Rally UXR and upload the list of user contacts to the project.

Sending an email wave

1. Using the percentages you calculated in the percentages tab of the USAT user list Google Sheet template, determine how many users for each plan type you need to contact for the wave.

If it's the first wave, use the population proportions. If it's a subsequent wave, use the proportions you calculate based on the responses so far (see next point).
2. To calculate your current sample plan proportions, download your survey results from Qualtrics and/or Rally. Calculate the percentage breakdown of plans so far. Then subtract that number from the population percentage and add the result to the population percentage.
An example with fake numbers:

Population percentage for Ultimate = 73%
Percentage of Ultimate plan types after sending wave 1 = 65%
Wave 2 percentage for Ultimate: (73% - 65%) + 73% = 81%
In this example, the sample is under the population, hence the next wave percentage is higher than the population to try to get within 3% of the population percentage for Ultimate.

3. Waves should be ~5,000 users each. Mark the desired number of users out of that 5,000 that fit your percentages for each plan type with the name of the wave you are sending.
4. In your Rally UXR project, filter out @gitlab.com email addresses, users who are on cooldowns/opted out of emails, and users contacted for previous USAT surveys from the past 12 months.
5. After filtering out those individuals, select the number of emails for your most recent wave.
6. Create a new email distribution using the USAT survey template email in Rally.
7. Send the email distribution. Typically emails are scheduled to go out Monday - Friday early in the morning US time (8 - 9am Eastern Time) with the goal of maximizing visibility and responses.

Once all email waves have been sent and you've hit the sample size goal for the quarter, add the user IDs and email addresses that were used this quarter to the previously contacted sheet, noting the quarter they were contacted. This allows us to avoid contacting the same users too frequently.

Outreach to USAT responders

When filling in the survey, UX Research gives USAT respondents the option to indicate whether they would be open to a follow up interview. As part of the analysis, the UX Research DRI will compile a list of users who agreed to the follow up interview (see Google Sheet template for USAT follow up users).

After the research report is shared out, the UX Research DRI will notify Product Managers, Product Designers, and Customer Success Managers about USAT responders who opted into contact via this USAT Responder Outreach issue template.md?reftype=heads). The USAT responder outreach workflow is described in more detail here.

Displaying the data

UX Research creates and communicates the USAT reports and partners with Product Data Insights to maintain the USAT dashboards for the UX department.

USAT dashboards and maintenance

There are two internal dashboards in Tableau meant for presenting USAT results:

USAT Scores: This dashboard shows the number and percentage of responses between those who are satisfied with GitLab the product (ratings of satisfied and very satisfied) and those who are not (ratings of neutral, dissatisfied, and very dissatisfied).
USAT Line Chart: This dashboard shows the USAT score on a quarterly basis in order to track our score over time.

These dashboards can also be found in the UX Department Performance Indicators page.

The survey data in the analysis template gets connected to Tableau to show data from the current and previous quarters. UX Research is responsible for working with Product Data Insights to update these Tableau dashboards each quarter.

Follow up questions

For past reports, UX Research has gotten requests to answer follow up questions about the USAT survey data (for example: What is the breakdown of company size across the survey responses?). When these requests arise, we've partnered with Product Data Insights to get support connecting USAT respondents to internal data sources (i.e., Snowflake). We create an ad hoc request issue in the Product Data Insights project in GitLab and submit this request to a member of their team.

Past USAT reports

FY25 reports

FY25 Q3
FY25 Q2

FY24 reports

FY24 Q3 Pilot

SECTION: product/ux/persona-creation/index.md

This handbook page defines the process for creating user personas only.

At GitLab, we use two kinds of personas that we create based on data-driven insights that focus on user needs and emotions:

- User personas - Used by UX professionals and Product Managers (PM) as a mechanism to connect with end users' needs, motivations, behaviors, and skills. Owned by Product Managers, who are also the DRI on persona-related research efforts.

- Buyer personas - Focus on the high-level goals of potential customers who may or may not be potential users. Owned by the Marketing team.

This insightful article by UX Collective goes into great detail on these differences and more.

User personas are developed to be useful to both Product Managers and Product Designers. They can be used to help understand the priority of potential features, increase the empathy in our communication and presentation to our users, and help shape design choices by understanding the background of our users. Each user persona is crafted to have a high amount of accuracy so that the information can remain useful for our stakeholders over a long period of time. Because of the importance of obtaining highly accurate information, the process of creating each persona is very detailed and can take a significant amount of time to research.

What makes up a user persona?

Each persona in GitLab's persona page should have the following traits:

- Job Summary - Should include major focus areas and a general description of skills and responsibilities
- Alternative Titles
- Motivations - It's suggested to keep to this format: When [situation], I want [feature] so that I can [task]
- Frustrations - A frustration should focus around a common blocker to their responsibilities
- Jobs to be Done (for recently updated personas)

(Optional)

- Personal Skills & Traits - Positive attributes that give life and empathy to your user persona
- Key Tools - Important software that helps the user complete their tasks
- Workflows- A series of steps the user will normally take to complete their tasks
- Collaborative Teams - Teams in their organiz